

DAG调度

项目背景

NP-Hard

DAG调度优化算法的研究进展

2. 主流DAG调度优化方法

2.1 禁忌搜索等启发式算法

2.2 混合整数线性规划（MILP）方法

2.3 基于图论的List Scheduling

2.4 强化学习方法

2.5 基于前瞻的树搜索算法

DAG图

优先级计算规则

常见的规则

启发式运筹算法中选取规则的动机

HEFT 算法（rank）

上行等级计算

动态EFT

最早开始时间（EST）

最早完成时间（EFT）

优先级计算规则

Look Ahead Tree Search 调度思路

目标

主要思路

伪代码

复杂度

对构造出的解进行Local Search

从trace中发现问题

构造结果中发现的gap

1次迭代的结果

20次迭代的结果

概述

算法组件

1. 调度表示

2. List<ScheduleResult>和SearchSolution之间的转换

局部搜索优化逻辑

搜索策略与原理

优化流程详解

伪代码

复杂度

结果

迭代前的结果

迭代后的结果

数值结果

实际运行结果

项目背景

随着深度学习模型规模的持续扩大，以GPT-4为代表的拥有数千亿参数的Transformer架构模型已远超单个GPU的内存容量与计算能力。在此背景下，分布式训练成为不可或缺的技术路径。然而，现有的并行策略——包括张量并行(TP)、序列并行(SP)、完全分片数据并行(FSDP)和专家并行(EP)——均面临GPU利用率不足的严峻挑战。Meta的研究报告揭示，大规模模型的FLOP利用率(MFU)往往低于43%，这主要源于分布式环境中的通信开销。具体而言，并行训练过程中的集体通信操作（如ReduceScatter和AllGather）占用了大量运行时间，导致计算资源大幅度闲置。

计算通信重叠技术被视为提升GPU利用率的关键策略。该技术通过在执行通信操作的同时进行计算处理，可显著缩短总体训练时间，最大化计算资源利用效率。本研究深入探讨了如何利用有向无环图(DAG)调度机制来优化这一重叠过程。在我们构建的模型中，计算图包含两类节点：计算节点与通信节点，同类型节点间存在互斥约束（即不能同时执行），这一特性进一步增加了调度问题的复杂性。

NP-Hard

我们的DAG调度框架首先将深度学习模型转换为中间表示(IR)，继而基于IR构造有向无环图，从而将节点间的重叠优化问题精确建模为基于约束的调度优化问题。本研究聚焦于如何在GPU上最优化地调度计算

图中的各个节点，同时综合考量节点间的依赖关系、资源消耗以及通信开销。尽管GPU本质上是并行处理器，但当我们关注特定类型的资源约束（如内存带宽限制）时，该问题可规约为经典单机调度问题的变体。在这种规约框架下，我们将GPU的关键受限资源视为单一“处理器”，而计算图的不同节点则竞争此资源。

值得注意的是，单机调度问题已被证明属于NP完全问题类别。Ullman在1975年的开创性研究中严格证明，即使在受限的条件下，寻找最优调度方案仍然是NP完全问题

(<https://www.sciencedirect.com/science/article/pii/S0022000075800080>)。鉴于我们的分布式训练调度问题包含更为复杂的约束条件和依赖关系，根据计算复杂性理论，该问题至少是NP难的。这一理论结果不仅解释了为何在实践中寻找计算图的全局最优调度如此具有挑战性，也为我们研究启发式算法和近似方法提供了坚实的理论基础。

DAG调度优化算法的研究进展

2. 主流DAG调度优化方法

2.1 禁忌搜索等启发式算法

禁忌搜索作为一种元启发式算法，在DAG调度问题中有着广泛应用。相比纯启发式调度策略，元启发式如禁忌搜索虽然时间开销较大，但能通过定向搜索解空间产生更高质量的结果。然而，对于大规模DAG图，这类算法面临巨大的搜索空间，其收敛时间往往过长，难以满足实际应用需求。其他元启发式算法包括蚁群优化(ACO)、遗传算法(GA)、模拟退火(SA)等，都存在类似的挑战。

2.2 混合整数线性规划（MILP）方法

MILP因其数学严谨性，理论上能找到最优解，在DAG调度中有着重要应用。研究者提出了包含可靠性和温度约束的MILP混合整数规划模型，用于最小化异构多核处理器平台上的执行时间。然而，正如先验分析所预期的，MILP求解时间随问题规模增长呈指数级增长，因此在实际应用中往往受限于问题规模，无法在短时间内对大规模数据进行有效求解。

2.3 基于图论的List Scheduling

List Scheduling作为一种经典的调度策略，基于特定优先级规则对任务进行排序。

HEFT(Heterogeneous Earliest Finish Time)和CPOP(Critical Path On a Processor)等算法是这类方法的代表 (<https://ieeexplore.ieee.org/document/993206>)。这些算法计算效率较高，适用于实时场

景，但由于仅利用了贪心启发式，往往陷入局部最优解，尤其是在节点间依赖关系复杂的情况下，调度质量有限。

2.4 强化学习方法

近年来，强化学习在调度领域展现出强大潜力。通过使用强化学习框架和专门设计的图神经网络 (GNN) 感知作业之间的拓扑依赖关系能够有效选择最有希望执行的任务。

(<https://arxiv.org/html/2103.06980>)

尽管强化学习方法有较好的前景，但是这种方法需要通过将图存到GNN中，由于我们问题的节点数量很多，因此面临诸多挑战：需要大量训练数据和时间，泛化能力有限，拓扑关系的存储占用大量内存，难以训练

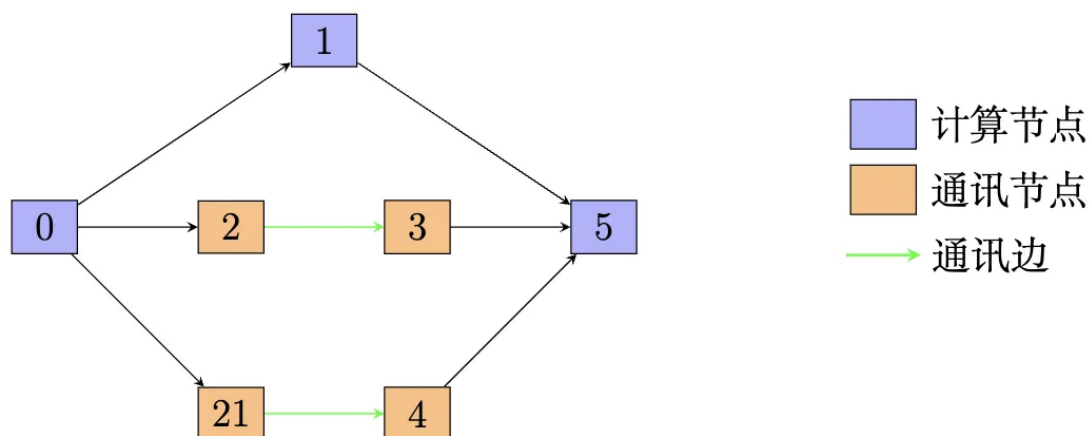
2.5 基于前瞻的树搜索算法

基于前瞻的树搜索算法作为一种构造式算法，能够在实际应用中快速得到高质量解决方案。这类算法通过探索可能的系统状态演化和后续动作产生的奖励序列，在计算预算用尽时输出最优行动。其核心思想是通过对搜索树的部分探索，对未来状态进行有限预测，从而在当前决策点做出更优的选择。

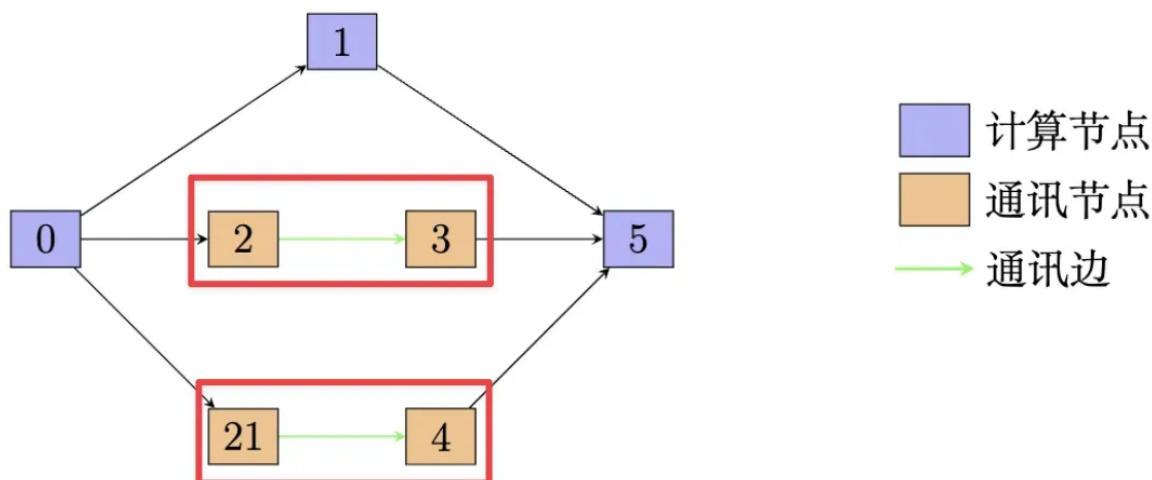
([https://openreview.net/forum?id=BUlkKOgaoQ&referrer=%5Bthe%20profile%20of%20Xihan%20Li%5D\(%2Fprofile%3Fid%3D~Xihan_Li1\)\)](https://openreview.net/forum?id=BUlkKOgaoQ&referrer=%5Bthe%20profile%20of%20Xihan%20Li%5D(%2Fprofile%3Fid%3D~Xihan_Li1))))

一种通用的基于前瞻的树搜索方法是寻找满足预算约束的树探索策略，使其在一组假设的初始状态上最大化平均回报。这类算法具有如下优势：(1)相比于完全搜索算法，大幅降低计算复杂度；(2)相比于纯启发式方法，能够提供更高质量的解；特别是，“最佳优先”策略通过线性组合前瞻路径特征参数化搜索策略，可针对特定问题定制高效的搜索算法。在DAG调度问题中，这种方法能够有效处理节点与节点间的复杂依赖关系和互斥约束。

DAG图



将 (communication node $\rightarrow$ communication edge $\rightarrow$ communication node) 合并成communication node进行处理，但是可能在2-3的edge期间，21节点可以执行，这样会造成时间的浪费，但是实际上，communication node的cost为1，因此这样处理不会造成时间上的浪费



优先级计算规则

常见的规则

1. 基于HEFT的Rank优先级：通过递归计算每个节点到终端节点的关键路径长度

2. **最早完成时间**：优先调度能够最早完成的任务
3. **拓扑层次优先**：优先调度拓扑层次较浅的任务
4. **基于后继节点数量的优先级**：优先调度有更多后继节点的任务
5. **基于内存效率的优先级**：优先调度那些执行时间相对于内存占用时间更短的节点
6. **内存释放潜力优先**：优先调度执行后能释放大量内存的节点
7. **内存影响最小化**：优先调度对系统内存影响最小的任务
8. **随机扰动**

启发式运筹算法中选取规则的动机

HEFT排序算法通常更适用于异构计算环境中的复杂工作流调度，其优势在于同时考虑任务执行时间和通信成本，能够有效处理复杂的任务依赖关系。HEFT在云计算、网格计算以及科学工作流等场景中表现出色，特别是当任务图具有复杂拓扑结构且计算资源异构性明显时。然而，HEFT的贪心策略和初始阶段的一次性rank计算可能导致在某些场景下产生次优调度，且对动态环境的适应性不足。

相比之下，EFT排序算法实现简单直观，计算效率高，专注于最小化任务完成时间。EFT更适合应用于实时系统、任务动态到达的在线调度环境以及依赖关系相对简单的任务调度场景。它在需要快速调度决策的环境中尤为有效，但往往只能实现局部优化而非全局优化，且可能忽略复杂的任务依赖网络。

heft更关注全局，即距离终点有多远，距离远的应该优先调度；因为eft考虑了依赖关系，因此eft更关注局部，计算这个节点最早什么时间能够执行完成，是一种很直接的贪心。（表述不太正式，但是是这个意思）

在我们的特定场景中，DAG图呈现出特殊的拓扑结构，这使得同时考虑HEFT和EFT的调度策略变得尤为必要和合理。对DAG图的节点进行观察，可以发现节点分布呈现出明显的聚集特征。一部分节点距离终点较近，这些节点虽然rank值可能比较接近，但由于它们之间存在复杂的依赖关系，导致它们的最早完成时间(eft)存在显著差异。另一部分节点距离终点较远，分布相对稀疏，这些节点大多没有前驱节点，因此它们的eft值相对接近，但由于到终点的距离不同，它们的rank值也各不相同。

这种节点分布特性充分说明了在调度过程中同时考虑rank和eft两个指标的重要性。通过结合这两个参数，我们能够在具有复杂拓扑结构的DAG图中更加精确地区分节点的调度优先级，从而提高整体调度效率。对于距离终点较近的节点簇，我们可以更多地依赖eft值来区分它们的执行顺序；而对于距离终点较远的节点，rank值则能够更好地反映它们对全局完成时间的影响。这种综合考虑策略既利用了HEFT的全局视角，又融合了EFT的即时性，能够更好地适应我们特定场景下DAG图的拓扑特性，实现更优的调度性能。

HEFT 算法 (rank)

HEFT (Heterogeneous Earliest Finish Time) 算法用于任务调度中，特别是在异构计算环境中，考虑了任务的计算成本和任务之间的依赖关系。HEFT 算法的核心是通过计算任务的 **上行等级** (Upward Rank) 来决定任务的调度顺序。上行等级衡量了任务完成所有后继任务所需要的总代价，优先调度上行等级高的任务。

from paper: <https://ieeexplore.ieee.org/document/993206>

上行等级计算

任务的 **上行等级** (Rank) 计算公式如下：

$$\text{Rank}(v) = \text{cost}(v) + \max_{\text{succ}(v)} (\text{Rank}(\text{succ}(v)))$$

其中， $\text{cost}(v)$ 表示任务 v 的计算成本， $\text{succ}(v)$ 是任务 v 的所有后继任务集合。即，任务的上行等级是它自身的计算成本与所有后继任务的最大上行等级之和。

动态EFT

动态EFT 计算考虑了任务的资源可用性，在计算最早开始时间时，不仅需要考虑任务的前驱任务，还需要考虑任务所需资源的可用时间。

最早开始时间 (EST)

任务的最早开始时间 (EST) 在动态EFT中计算时，除了考虑前驱任务的最早完成时间，还要考虑任务所需资源的最早可用时间：

$$\text{EST}(v) = \max \left(\max_{\text{pred}(v)} (\text{EFT}(\text{pred}(v)) + \text{cost}(\text{pred}(v), v)), \text{available_time}(v) \right)$$

其中， $\text{available_time}(v)$ 表示任务 v 所需资源的最早可用时间。

最早完成时间 (EFT)

动态EFT的最早完成时间 (EFT) 计算与静态EFT相同，都是通过最早开始时间 (EST) 加上任务本身的计算成本：

$$\text{EFT}(v) = \text{EST}(v) + \text{cost}(v)$$

但动态EFT中，最早开始时间的计算会受到资源可用性的影响。

优先级计算规则

$$\text{priority} = \text{rank} - \text{EFT}$$

该计算方式结合了节点的上行等级（即节点本身和所有后继任务的执行成本总和）与节点的最早完成时间（EFT）。优先调度那些上行等级较高且最早完成的节点。

Look Ahead Tree Search 调度思路

目标

在经典的 DAG 调度算法中，通常会基于某种优先级（例如 $\text{rank} - \text{EFT}$ ）来在就绪节点中一次只选一个最高优先级的节点进行调度，然后循环往复，直到所有节点被调度完毕。

然而，这种"逐步选择一个节点"的贪心策略在某些情况下并不一定获得最优的全局调度结果。为了在局部拓宽搜索范围，可以考虑在每次决策中选择 Top-K 个优先级最高的候选节点分别尝试，然后对每个候选节点进行一次"有限深度的补全模拟"，最终比较出最优解后再落地真实调度。

主要思路

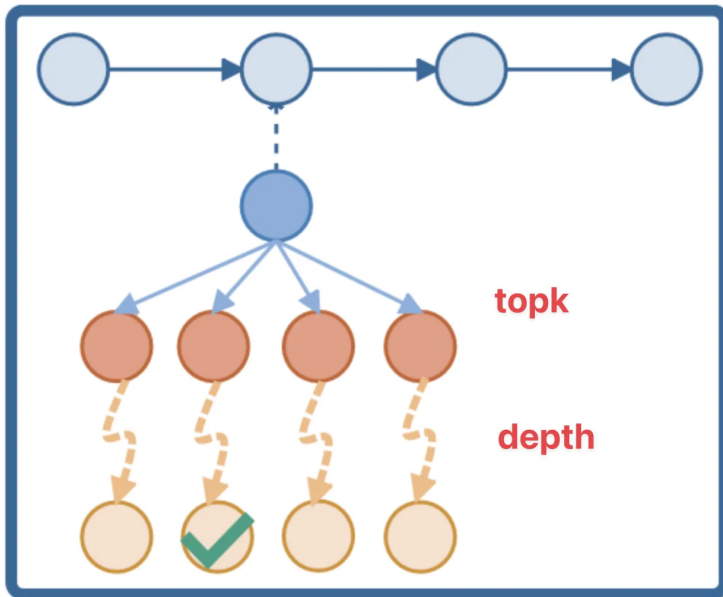
(1) Top-K 候选节点：

- 在某一步调度中，对所有入度为 0（就绪）的节点计算其优先级（如 $\text{priority} = \text{rank} - \text{EFT}$ ）。
- 将它们按优先级从大到小排序，选出前 K 个候选节点；若就绪节点数小于 K ，则全部取用。

(2) 有限深度的补全模拟 (`simulateGreedySchedule`)：

- 对每个候选节点，拷贝当前的所有调度状态（`startTime`、`finishTime`、`availableTime`、`unscheduledNodes`、`inDegree` 等），然后先"假设"本步选的就是它。
- 记录下调度前的初始 `makespan`。
- 在这个假设状态下，不再是将所有剩余未调度节点全部模拟完成，而是仅模拟固定深度 `depth` 的节点，按固定的贪心策略（同样 $\text{priority} = \text{rank} - \text{EFT}$ 或其他）进行排序。
- 模拟过程中追踪已调度节点的总成本 `totalCostOfScheduledNodes`。

- 计算"效率比率": $ratio = \frac{finalMakespan - initialMakespan}{totalCostOfScheduledNodes + currentNodeCost}$, 该指标表示单位成本能够带来的时间增长, 值越小越好, 如果ratio一样, 则优先选择后继节点更多的节点调度



(3) 零成本节点与特殊情况处理:

- 零成本节点 (`cost` = 0) 在模拟中优先处理, 但不计入深度限制和总成本。
- 当没有可调度的非零成本节点时, 返回一个合理的中等效率比率值, 避免极端值干扰决策。
- 设置最大迭代次数限制, 防止潜在的死锁情况导致无限循环。

(4) 并行化:

- 因为对每个候选节点的"补全模拟"是完全独立的, 为了加速, 使用多线程池 (`ExecutorService`) 并行地执行这些模拟任务。
- 代码中使用 `executor.invokeAll()` 提交多个 `Callable` , 分别返回 `(candidate, efficiencyRatio)` 这样的结果。
- 主线程收集所有结果后, 选出效率比率最小者对应的候选节点为最终决定的节点。

(5) 更新真实调度:

- 只有那一个"效率比率最优"的候选节点会被真实地加入调度结果, 更新到真正的 `startTime` 、 `finishTime` 、 `availableTime` 、 `unscheduledNodes` 、 `inDegree` 中。
- 其余候选节点的结果仅用于评估, 不会写回全局数据结构。

内存使用分析 <https://yuque.antfin.com/tangshenghao.tsh/kb/kgrrn06gpstm8wlag>

伪代码

▼ 更加简洁

Java |

```
1  Algorithm HybridScheduler
2  Input:  $G(V, E)$  (DAG)
3  Output: Schedule  $S$ 
4   $UR \leftarrow \text{ComputeUpwardRanks}(G)$ 
5   $U \leftarrow V$  // Unscheduled nodes
6  While  $U \neq \emptyset$  do
7       $Z \leftarrow \{n \in U \mid \text{Pred}(n) \subseteq (V \setminus U) \wedge \text{cost}(n) = 0\}$ 
8      Schedule all nodes in  $Z$  and update  $U$ 
9       $R \leftarrow \{n \in U \mid \text{Pred}(n) \subseteq (V \setminus U) \wedge \text{cost}(n) > 0\}$ 
10     If  $R = \emptyset$  then continue
11     For each  $n \in R$  do
12          $\text{EST}(n) \leftarrow \text{ComputeEarliestStartTime}(n)$ 
13          $\text{Priority}(n) \leftarrow UR(n) - (\text{EST}(n) + \text{cost}(n))$ 
14      $C \leftarrow \text{TopK}(R, \text{Priority})$ 
15     For each  $c \in C$  in parallel dos
16          $\text{SimSchedule} \leftarrow \text{SimulateGreedySchedule}(c, d)$  //d is simulation depth
17          $\text{ER}(c) \leftarrow (\text{SimSchedule.makespan} - \text{CurrentMakespan}) / \text{TotalCost}$ 
18      $b \leftarrow \text{argmin}_{c \in C}(\text{ER}(c))$ 
19     Schedule  $b$  at  $\text{EST}(b)$ 
20      $U \leftarrow U \setminus \{b\}$ 
21 Return  $S$  sorted by start time
```

```

1  Algorithm HybridScheduler(G)
2      UR ← ComputeUpwardRanks(G) // Critical path-based priority values
3      Initialize data structures and unscheduled set U ← V
4
5      While U ≠ ∅ do
6          // First process all ready zero-cost nodes
7          Schedule all ready zero-cost nodes and update U
8
9          // Process non-zero cost nodes
10         R ← {ready nodes with cost > 0}
11         If R = ∅ then continue
12
13         // Compute priorities and times
14         For each node n ∈ R:
15             EST(n) ← ComputeEarliestStartTime(n)
16             Priority(n) ← UR(n) - (EST(n) + cost(n))
17
18         // Select top-K candidates based on EST and priority
19         C ← Top-K nodes from R sorted by Priority
20
21         // Parallel evaluation through simulation
22         For each candidate c ∈ C in parallel:
23             Schedule c
24             SimulateGreedySchedule(depth d)
25             ER(c) ← (SimulatedMakespan - CurrentMakespan) / TotalCost
26
27         // Select best candidate and update schedule
28         Select node b ∈ C with minimum ER
29         Schedule b at EST(b) and update U
30
31     Return S sorted by start time

```

复杂度

$$O(n + m) + O(n) \cdot [O(n) + O(k \log k) + O(k \cdot n)] + O(n \log n) = O(n^2 * k)$$

k为topK，即算法参数中设置的常数，因此算法复杂度为 $O(n^2)$

Algorithm HybridScheduler(G)

UR $\leftarrow$ ComputeUpwardRanks(G) // Critical path-based priority values $O(n+m)$

Initialize data structures and unscheduled set $U \leftarrow V$

While $U \neq \emptyset$ do $O(n)$

// First process all ready zero-cost nodes $O(n)$

Schedule all ready zero-cost nodes and update U

// Process non-zero cost nodes $O(n)$

$R \leftarrow \{\text{ready nodes with cost} > 0\}$

If $R = \emptyset$ then continue

// Compute priorities and times $O(n)$ (worst case)

For each node $n \in R$:

EST(n) $\leftarrow$ ComputeEarliestStartTime(n)

Priority(n) $\leftarrow$ UR(n) - (EST(n) + cost(n))

// Select top-K candidates based on EST and priority $O(k \log k)$

$C \leftarrow$ Top-K nodes from R sorted by Priority

// Parallel evaluation through simulation $O(k*n)$

For each candidate $c \in C$ in parallel:

Schedule c

SimulateGreedySchedule(depth d)

ER(c) $\leftarrow$ (SimulatedMakespan - CurrentMakespan) / TotalCost

// Select best candidate and update schedule

Select node $b \in C$ with minimum ER

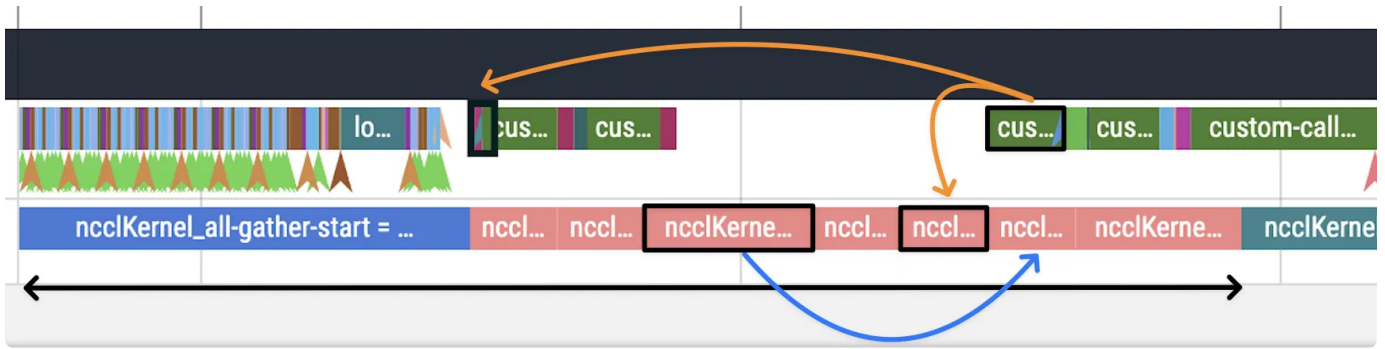
Schedule b at EST(b) and update U

Return S sorted by start time $O(n \log n)$

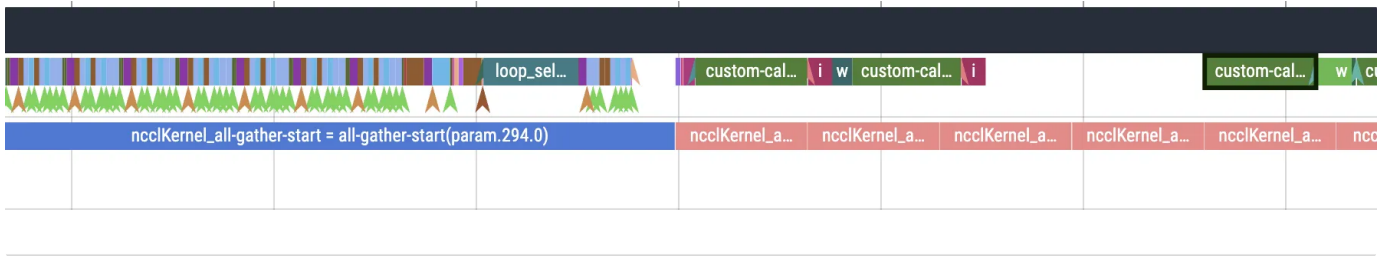
对构造出的解进行Local Search

从trace中发现问题

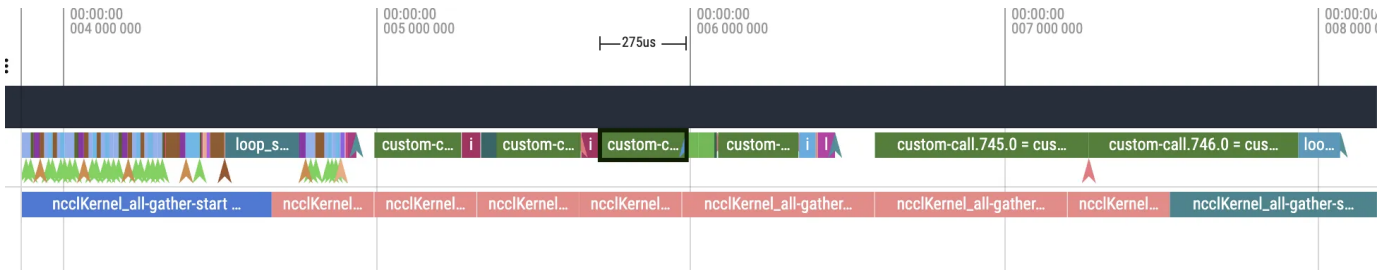
构造结果中发现的gap



1次迭代的结果



20次迭代的结果



概述

该算法通过对communication节点做微小扰动后再重新排序来最小化整体执行时间(makespan)，同时保持依赖关系约束。

算法组件

1. 调度表示

算法使用两种主要表示形式：

- `List<ScheduleResult>`：表示已调度节点的开始和结束时间

```

1 public class ScheduleResult {
2     public String uuid;
3     public String name;
4     public String typename;
5     public int startTime;
6     public int finishTime;
7 }

```

- **SearchSolution**: 将节点类型映射到节点UUID的有序列表，定义执行优先级

```

1 public class SearchSolution {
2     public Map<String, List<String>> solution;
3 }

```

2. List<ScheduleResult>和SearchSolution之间的转换

List<ScheduleResult> toScheduleResults(SearchSolution _solution, DAGGraph graph)

在不违反以下约束的情况下，快速构造makespan最小的解

- 前驱节点完成时间
- 节点间的通信成本
- 资源可用性约束

SearchSolution toSearchSolution(List<ScheduleResult> results)

对results进行一次遍历

局部搜索优化逻辑

搜索策略与原理

1. 关键路径识别:

- 算法识别出对整体makespan影响最大的"communication"类型节点
- 这些节点往往构成了调度的关键路径或瓶颈

2. 邻域定义:

- 对于每个选定的通信节点，其"邻域"被定义为当前位置前后 `Parameter.localSearchSize` 范围内的所有可能位置
- 这种有限邻域设计既保证了探索的多样性，又控制了计算复杂度

3. 增量评估：

- 通过移动单个节点并重新计算调度，实现对调度品质的增量评估
- 每次尝试的变动量小，保证了搜索空间的连续性

优化流程详解

`iterativeOptimize` 方法中的局部搜索优化流程可详细分解为：

1. 初始解生成：

- 以输入的调度结果作为初始解
- 计算初始makespan作为优化基准

2. 候选节点选择：

- 选择localSearchSize范围内的节点进行交换

3. 并行邻域探索：

- 对每个候选位置创建一个CompletableFuture异步任务
- 在每个任务中：
 - 创建新的节点排序（移除原位置节点并插入新位置）
 - 构建新的SearchSolution对象
 - 转换为具体调度并计算makespan
 - 返回结果包含新调度和对应makespan

4. 解评估与选择：

5. 比较所有有效调度方案的makespan，选择具有最小makespan的方案作为候选改进

5. 解更新机制：

- 如果找到更优解（makespan更小），立即更新最佳解记录，如果一次迭代没有更新解，就结束迭代。

伪代码

```
1 Algorithm DAGLocalSearch
2 Input: G (DAG), S (initial schedule), I (max iterations)
3 Output: Improved schedule S'
4
5 S' ← S
6 bestMakespan ← CalculateMakespan(S')
7 for i = 1 to I do
8     C ← FindNodesNearBubbles(S') // 寻找当前解中存在bubble的位置附近的节点
9     if C = ∅ then break
10    improved ← false
11    for n ∈ C at position p do
12        for q ∈ N(p,w) in parallel do // N(p,w): neighborhood of p with
window size w
13            if q = p then continue
14            C' ← Move(C, n, p→q)
15            S'' ← UpdateSchedule(S', C')
16            makespan ← CalculateMakespan(S'')
17            if makespan < bestMakespan then
18                S' ← S''
19                bestMakespan ← makespan
20                improved ← true
21    if improved then break
22    if ¬improved then break
23 return S'
```

```

1  Algorithm DAGLocalSearch(G, S, I)
2      S' ← S
3      BestMakespan ← CalculateMakespan(S')
4
5      For iter = 1 to I do
6          Improved ← false
7
8          // Convert to solution representation
9          SOL ← ToSearchSolution(S')
10
11         // Focus on communication nodes
12         CommNodes ← SOL["communication"]
13         If CommNodes = ∅ then break
14
15         // Try moving each node to different positions
16         For each node n at position i in CommNodes do
17             // Parallel search within neighborhood
18             For newPos from max(0, i-WindowSize) to min(|CommNodes|-1, i+Win
indowSize) in parallel do
19                 If newPos = i then continue
20
21                 // Create new ordering by moving node n to newPos
22                 NewCommNodes ← CommNodes with n moved from i to newPos
23                 NewSOL ← SOL with CommNodes replaced by NewCommNodes
24
25                 // Generate and evaluate new schedule
26                 NewS ← ToScheduleResults(NewSOL, G)
27                 NewMakespan ← CalculateMakespan(NewS)
28
29                 // Update if better solution found
30                 If NewMakespan < BestMakespan then
31                     S' ← NewS
32                     BestMakespan ← NewMakespan
33                     Improved ← true
34
35                 If Improved then break
36
37         // No improvement in this iteration, terminate early
38         If not Improved then break
39
40     Return S'
41

```

复杂度

$$O(n) + O(I) \cdot [O(n) + O(n) \cdot O(w) \cdot O(n + m)] = O(I * n^2 * w)$$

I 为迭代次数, w为localSearch的搜索范围, 均为常数, 因此复杂度为 $O(n^2)$

```
Algorithm DAGLocalSearch(G, S, I)
  S' ← S
  BestMakespan ← CalculateMakespan(S')      O(n)

  For iter = 1 to I do      O(I)
    Improved ← false

    // Convert to solution representation
    SOL ← ToSearchSolution(S')      O(n)

    // Focus on communication nodes
    CommNodes ← SOL["communication"]
    If CommNodes = ∅ then break

    // Try moving each node to different positions
    For each node n at position i in CommNodes do      O(n)
      // Parallel search within neighborhood
      For newPos from max(0, i-WindowSize) to min(|CommNodes|-1, i+WindowSize)
        If newPos = i then continue      O(w)

        // Create new ordering by moving node n to newPos
        NewCommNodes ← CommNodes with n moved from i to newPos
        NewSOL ← SOL with CommNodes replaced by NewCommNodes

        // Generate and evaluate new schedule
        NewS ← ToScheduleResults(NewSOL, G)      O(n + m)
        NewMakespan ← CalculateMakespan(NewS)

        // Update if better solution found
        If NewMakespan < BestMakespan then
          S' ← NewS
          BestMakespan ← NewMakespan
          Improved ← true

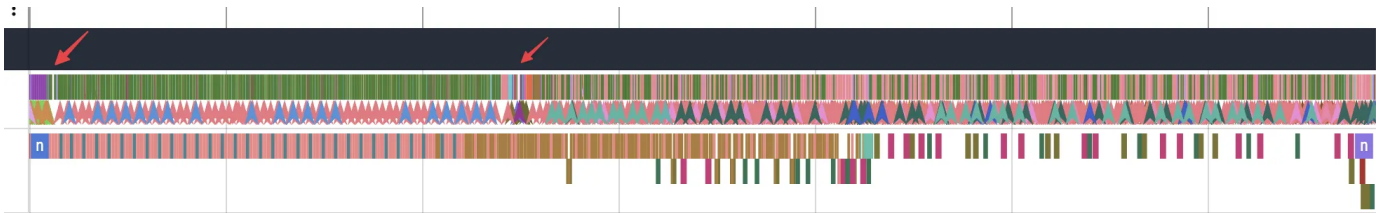
      If Improved then break

    // No improvement in this iteration, terminate early
    If not Improved then break

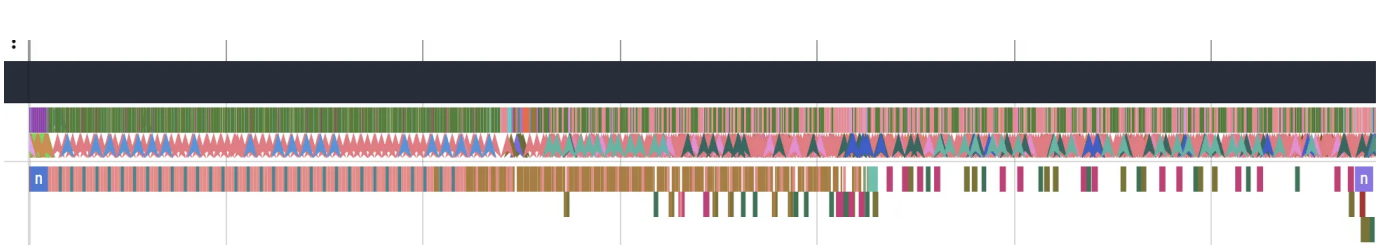
  Return S'
```

结果

迭代前的结果



迭代后的结果



数值结果

treeSearch	treeSearch + localSearch
342859	341662
520054	519881
513951	513101
348398	347762

实际运行结果

0317是tree search

0320是tree search + local search

配置	baseline (XLA)	ortools	0317	0320	Megatron (h800/989TFLOPS)
TP=1 DP=8	0.61s/iter 40.95GB	0.52s/iter 48.31GB	0.48s/iter 49.26GB	0.47s/iter 50.12GB	0.51s 79GB 405TFLOPS
TP=2 DP=4	0.69s/iter 40.97GB	0.58s/iter 45.18GB	0.58s/iter 45.18GB	0.47s/iter 55.75GB	0.52s 60GB 400TFLOPS
TP=4 DP=2	0.69s/iter 40.94GB	0.69s/iter 45.2GB	0.58s/iter 45.14GB	0.47s/iter 54.22GB	0.565s 50GB 367TFLOPS
TP=8 DP=1	0.57s/iter 44.24GB	0.54s/iter 41.74GB	0.48s/iter 48GB	0.50s/iter 49.87GB	0.8s 47GB 259TFLOPS